

Integrantes:

Amarantha Soto V-25153461

Marcos Useche V-24349232

Luis Colmenares V-26036875

Documentación del Código — Cajero Automático Basado en Autómatas**Descripción General**

Este programa implementa un sistema de **cajero automático (ATM)** utilizando el concepto de **Autómatas Finitos Deterministas (DFA)**.

El sistema está compuesto por dos clases principales:

1. BancoDFA

- Simula el comportamiento del banco.
- Gestiona tarjetas, PIN, saldo y bloqueo de tarjetas.

2. CajeroAutomaticoDFA

- Implementa el autómata del cajero automático.
- Controla los estados, transiciones y flujo de la operación.

El programa permite:

- Validar tarjetas
- Validar PIN
- Consultar saldo
- Retirar dinero
- Cambiar clave
- Bloquear tarjetas tras múltiples intentos fallidos

Importaciones

import time

Explicación

Se utiliza el módulo time para generar pausas con time.sleep() y simular tiempos reales de comunicación entre el cajero y el banco.

Clase BancoDFA

class BancoDFA:

Propósito

Representa el sistema bancario encargado de validar y procesar operaciones.

Constructor

def __init__(self):

Variables principales

self.estado_actual = "b0"

Estado actual del banco.

Base de datos simulada

```
self.tarjetas = {  
    "123456": {  
        "pin": "1234",  
        "saldo": 1000,  
        "bloqueada": False  
    }  
}
```

Contiene:

- Número de tarjeta
- PIN
- Saldo
- Estado de bloqueo

Métodos de BancoDFA

validar_tarjeta()

def validar_tarjeta(self, numero_tarjeta):

Función

Verifica si la tarjeta existe y si no está bloqueada.

Parámetros

- numero_tarjeta: número de tarjeta ingresado.

Retorna

(True, "Tarjeta válida")

o

(False, "Tarjeta inválida")

validar_pin()

```
def validar_pin(self, numero_tarjeta, pin):
```

Función

Compara el PIN ingresado con el almacenado.

Retorna

- True → PIN correcto
- False → PIN incorrecto

verificar_fondos()

```
def verificar_fondos(self, numero_tarjeta, monto):
```

Función

Comprueba si el saldo es suficiente para realizar el retiro.

descontar_saldo()

```
def descontar_saldo(self, numero_tarjeta, monto):
```

Función

Resta el monto retirado del saldo disponible.

consultar_saldo()

```
def consultar_saldo(self, numero_tarjeta):
```

Función

Devuelve el saldo actual de la cuenta.

cambiar_pin()

```
def cambiar_pin(self, numero_tarjeta, nuevo_pin):
```

Función

Actualiza el PIN del usuario.

bloquear_tarjeta()

```
def bloquear_tarjeta(self, numero_tarjeta):
```

Función

Bloquea la tarjeta después de múltiples intentos fallidos.

Clase CajeroAutomaticoDFA

```
class CajeroAutomaticoDFA:
```

Propósito

Representa el autómata del cajero automático.

Implementa formalmente:

```
[  
M = (Q, \Sigma, \delta, q0, F)  
]
```

Donde:

- Q = conjunto de estados
- Σ = alfabeto de entrada
- δ = función de transición
- q_0 = estado inicial
- F = estados finales

Constructor del Cajero

```
def __init__(self):
```

Estados del Autómata (Q)

```
self.Q = {  
    "q0", "q1", ..., "q14"  
}
```

Estados principales

Estado Descripción

q0	Reposo
q1	Tarjeta insertada
q2	Validando tarjeta
q3	Tarjeta válida
q4	Validando PIN
q5	Usuario autenticado
q6	Menú principal
q7	Retiro
q8	Verificando fondos
q9	Entregando dinero
q10	Consulta saldo
q11	Cambio de clave
q12	Transacción finalizada
q13	Error
q14	Tarjeta bloqueada

Alfabeto (Σ)

```
self.Sigma = {  
    ...  
}
```

Representa los símbolos o eventos que producen transiciones.

Ejemplos:

- insertar_tarjeta
- validar_tarjeta
- seleccionar_retiro
- seleccionar_consulta
- finalizar

Estado Inicial

```
self.estado_actual = "q0"
```

El sistema comienza en reposo.

Estados Finales

```
self.F = {"q12", "q14"}
```

Significado

- q12 → operación terminada
- q14 → tarjeta bloqueada

Función de Transición δ

```
def transicion(self, simbolo):
```

Propósito

Controla el cambio de estados del autómata dependiendo del símbolo recibido.

Flujo del Sistema

1. Inserción de Tarjeta

$q0 \rightarrow q1$

El usuario ingresa una tarjeta.

2. Validación de Tarjeta

$q1 \rightarrow q2 \rightarrow q3$

El cajero consulta al banco para validar la tarjeta.

Si falla:

$q1 \rightarrow q13$

3. Validación de PIN

$q3 \rightarrow q4 \rightarrow q5$

Si el PIN es incorrecto:

$q3 \rightarrow q13$

Después de 3 intentos:

$q14$

La tarjeta queda bloqueada.

4. Menú Principal

$q5 \rightarrow q6$

Opciones disponibles:

1. Retiro
2. Consulta saldo
3. Cambio clave
4. Salir

5. Retiro de Dinero

$q6 \rightarrow q7 \rightarrow q8 \rightarrow q9 \rightarrow q12$

Proceso

- Solicita monto
- Verifica fondos
- Descuenta saldo
- Entrega dinero

Si no hay fondos:

q13

6. Consulta de Saldo

q6 → q10 → q12

Muestra el saldo disponible.

7. Cambio de Clave

q6 → q11 → q12

Actualiza el PIN del usuario.

8. Manejo de Errores

q13

El sistema pregunta si desea reintentar.

Método ejecutar()

```
def ejecutar(self):
```

Función

Simula el funcionamiento completo del cajero automático.

Controla:

- Entrada de datos
- Navegación entre estados
- Ejecución de transiciones

Ejecución Principal

```
if __name__ == "__main__":
```

Función

Punto de inicio del programa.

```
cajero = CajeroAutomaticoDFA()
```

```
cajero.ejecutar()
```


Crea una instancia del cajero y ejecuta el sistema.

Características Técnicas

Paradigma Utilizado

- Programación Orientada a Objetos (POO)
- Autómatas Finitos Deterministas (DFA)

Ventajas del Diseño

Modularidad

El banco y el cajero están separados.

Escalabilidad

Se pueden agregar fácilmente:

- Transferencias
- Depósitos
- Múltiples usuarios
- Base de datos real

Seguridad

Implementa:

- Validación de PIN
- Bloqueo de tarjeta
- Control de errores

Ejemplo de Uso

Tarjeta válida

Tarjeta: 123456

PIN: 1234

Resultado

- Acceso permitido
- Menú principal habilitado

PIN incorrecto 3 veces

Resultado

[ATM] TARJETA BLOQUEADA